

WEEK-11

Task-1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor 4 Components

```
root@Mani:~/skill$ ls
README.md      input.txt      pwd_exit
append.txt     input_redirect pwd_exit.c
append_redirect input_redirect.c result.txt
append_redirect.c job_table      resume
background     job_table.c    resume.c
background.c    jobtable       sigint_shell
cd_command     jobtable.c     sigint_shell.c
cd_command.c    mini_shell     signal_handlers
combined_redirect mini_shell.c   signal_handlers.c
combined_redirect.c multiple_pipes  signals
complex_command multiple_pipes.c signals.c
complex_command.c output.txt      sigtstp
docs           output_redirect sigtstp.c
error.txt      output_redirect.c stderr_redirect
export_command pipe             stderr_redirect.c
export_command.c pipe.c          test.txt
foreground     pipeline_structure test_report.txt
foreground.c    pipeline_structure.c test_shell.sh
history        process_group    tests
history.c      process_group.c

root@Mani:~/skill$ gcc jobtable.c -o jobtable
root@Mani:~/skill$ ^C
root@Mani:~/skill$ gcc background.c -o background
root@Mani:~/skill$ gcc sigint_shell.c -o sigint_shell
root@Mani:~/skill$ ./jobtable
Job ID  PID    Status    Command
-----
[1]     1234    Running   sleep 30
[2]     1235    Stopped   sleep 50
root@Mani:~/skill$ ./background
Starting background job...
Background job started with PID: 1839
Prompt returned immediately.
Job is still running.
^C
root@Mani:~/skill$ ./process_group
Parent PID : 1850
Child PID  : 1851
Child PGID : 1851
Child PID  : 1851
```

Report: The different modules developed during the practicals were compiled and tested individually.

Their interfaces and execution behavior were checked to ensure that the components work

correctly.

Error conditions were also tested and the code was refined where required.

This integration testing improved the reliability and overall organization of the project.

Task-2: TO Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

Code:

```
GNU nano 8.7.1 error_test.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;

    printf("Testing file error...\n");

    fp = fopen("missing.txt", "r");

    if (fp == NULL) {
        perror("Error opening file");
    } else {
        printf("File opened successfully.\n");
        fclose(fp);
    }

    printf("Program continues after error.\n");

    return 0;
}
```

Output:

```
root@Mani:~/skill$ nano error_test.c
root@Mani:~/skill$ gcc error_test.c -o error_test
root@Mani:~/skill$ ./error_test
Testing file error...
Error opening file: No such file or directory
Program continues after error.
root@Mani:~/skill$ |
```

Report: Runtime error handling was tested using invalid file and command conditions. The program generated meaningful error messages using standard error-handling functions. The program continued execution after handling the error instead of terminating unexpectedly. Different error scenarios were tested to verify reliable recovery.

Task-3: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

Code:

```
GNU nano 8.7.1                                memory_test.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *data = malloc(10 * sizeof(int));

    if (data == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 10; i++)
        data[i] = i;

    printf("Memory allocated successfully.\n");

    free(data);

    return 0;
}
```

Output:

```
root@Mani:~/skill$ nano memory_test.c
root@Mani:~/skill$ gcc -g memory_test.c -o memory_test
root@Mani:~/skill$ ./memory_test
Memory allocated successfully.
root@Mani:~/skill$ valgrind --leak-check=full ./memory_test
==3094== Memcheck, a memory error detector
==3094== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==3094== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==3094== Command: ./memory_test
==3094==
Memory allocated successfully.
==3094==
==3094== HEAP SUMMARY:
==3094==   in use at exit: 0 bytes in 0 blocks
==3094==   total heap usage: 2 allocs, 2 frees, 1,064 bytes allocated
==3094==
==3094== All heap blocks were freed -- no leaks are possible
==3094==
==3094== For lists of detected and suppressed errors, rerun with: -s
==3094== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@Mani:~/skill$
```

Report: Valgrind was used to check the program for memory leaks and invalid memory operations.

The dynamically allocated memory was properly released using free().

The program was tested with Valgrind to verify that the allocated resources were cleaned up correctly.

The final results were checked to confirm that no memory leaks or memory errors remained.

Task-4: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

Code:

```
GNU nano 8.7.1 test_project.sh *
#!/bin/bash

echo "===== PROJECT TESTING ====="

echo "Test 1: Job Table"
./jobtable

echo
echo "Test 2: Process Group"
./process_group

echo
echo "Test 3: Error Handling"
./error_test

echo
echo "Test 4: Memory Test"
./memory_test

echo
echo "===== TESTING COMPLETED ====="
```

Output:

```
root@Mani:~/skill$ nano test_project.sh
root@Mani:~/skill$ chmod +x test_project.sh
root@Mani:~/skill$ ./test_project.sh
===== PROJECT TESTING =====
Test 1: Job Table
Job ID  PID      Status      Command
-----
[1]     1234     Running     sleep 30
[2]     1235     Stopped     sleep 50

Test 2: Process Group
Parent PID : 3132
Child PID  : 3133
Child PGID : 3133
Child PID  : 3133
Process Group ID: 3133

^C^X
root@Mani:~/skill$ ./test_project.sh > test_report.txt

^C
root@Mani:~/skill$ cat test_report.txt
===== PROJECT TESTING =====
Test 1: Job Table
Job ID  PID      Status      Command
-----
[1]     1234     Running     sleep 30
[2]     1235     Stopped     sleep 50

Test 2: Process Group
Child PID  : 3148
Process Group ID: 3148
root@Mani:~/skill$
```

Report: Test cases were created to verify the functionality of the developed modules. A shell script was used to automate the execution of multiple programs and collect their outputs. The actual results were compared with the expected behavior to identify possible errors. A test report was generated to document the final testing results and verify project functionality.